## PROCEDURE

1. Click on XILINX VIVADO2014.2
2. Open a new Project and give name to theproject.
3. Open a new source file and give name and specify input and output portnames.
4. Type Verilog code and save thefile.
5. Compile sourcefile.
6. Add test bench to the main module and specify the test vectors init.
7. Simulate the test bench and note down the simulationresults.
8. Synthesize the main module and note down the RTLSchematic.
9. Elaborate the design and assign the pins to thei/os
10. Perform Implementation which internally performs floor planning, placement androuting
11. Generate a bit stream for the givenprogram
12. Dump the bit file on the FPGA and check itsfunctionality
13. Verify the functionality using the Logicanalyzer.

## HDL PROGRAM

```
module MUX1 (
 input [3:0] sel,
 input A,B,C,D,E,F,G,H,G,H,I,J,K,L,M,N,O,P,
 Output q);
 reg q;
 always @( A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,
          sel)
 begin
  Case(sel)
  4'd0: q= A;
  4'd1: q = B;
  4'd2: q = C;
  4'd3: q = D;
  4'd4: q = E;
  4'd5: q = F
  4'd6: q = G
  4'd7: q = H
  4'd8: q = I
  4'd9: q = J
  4'd10: q = K
  4'd11: q = L
  4'd12: q = M
  4'd13: q = N
  4'd14: q = O
  4'd14: q = P
  default: ;
  end Case
  end
```

```
module mux16x1_tb;
reg A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P;
reg [3:0] sel;
wire q;
mux16x1
 out(.A(A),.B(B),.C(C),.D(D),.E(E),.F(F),.G(G),.H(H),
 .I(I),.J(J),.K(K),.L(L),.M(M),.N(N),
 .O(O),.P(P),.sel(sel),.q(q));
initial begin
sel=4'b0000; A=1; B=0; C=0; D=0; E=0; F=0; G=0; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100;
sel=4'b0001; A=0; B=1; C=0; D=0; E=0; F=0; G=0; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100;
sel=4'b0010; A=0; B=0; C=1; D=0; E=0; F=0; G=0; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100;
sel=4'b0011; A=0; B=0; C=0; D=1; E=0; F=0; G=0; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100;
sel=4'b0100; A=0; B=0; C=0; D=0; E=1; F=0; G=0; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100
sel=4'b0101; A=0; B=0; C=0 D=0; E=0; F=1; G=0; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100
sel=4'b0110; A=0; B=0; C=0; D=0; E=0; F=0; G=1; H=0; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100
sel=4'b0111; A=0; B=0; C=0; D=0; E=0; F=0; G=0; H=1; I=0; J=0;
    K=0; L=0; M=0; N=0; O=0; P=0; #100
sel=4'b1000; A=0; B=0; C=0; D=0; E=0; F=0; G=0; H=0; I=1; J=0;
```

**HDL PROGRAM**

```
module full-adder(A,B,cin,out,cout)
 input A,B,Cin;
 output out, Cout;
 assign out ,Cout;
 assign Cout = A^B^Cin;
 assign Cout = ((A&B)|(B&Cin)|(A&Cin));
 end module
module ripple-carry-adder(A,B,cin,out,cout);
 input [3:0] A;          full_adder FA0[A[0],B[0],out[0]
 input [3:0]B;           full-adder FA1[A[1],B[1],C1,      C1);
 input Cin;                                  out[1],C2);
 output [3:0] out;       full-adder FA2(A[2],B[2],C2,out[2],C3);
 output Cout             full-adder FA3[A[3],B[3],C3,out[3],
 wire C1,C2,C3;          end module.                       Cout);
```

**TESTBENCH**

```
module ripple-carry-adder_tb;
        reg [3:0]A;
        reg [3:0]B;

   reg Cin;
      wire [3:0] out;
      wire Cout;
ripple_carry-adder uut(.A(A),.B(B),.Cin(Cin),.out(out),
                .Cout(Cout));

initial begin
        Cin=0;
        A= 4'b0000 ; B=4'b0000; #10;
        A= 4'b0000 ; B=4'b0001; #10;
        A= 4'b0000 ; B=4'b010; #10;
        A= 4'b0000; B=4'b0011; #10;
        end
```

**TRUTH TABLE**

| A | B | C | SUM | CARRY |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

**PROCEDURE**

1. Click on XILINX VIVADO2014.2
2. Open a new Project and give name to the project.
3. Open a new source file and give name and specify input and output portnames.
4. Type Verilog code and save the file.
5. Compile source file.
6. Add test bench to the main module and specify the test vectors init.
7. Simulate the test bench and note down the simulation results.
8. Synthesize the main module and note down the RTL Schematic.
9. Elaborate the design and assign the pins to thei/os
10. Perform Implementation which internally performs floor planning, placement and routing
11. Generate a bit stream for the given program
12. Dump the bit file on the FPGA and check its functionality
13. Verify the functionality using the Logic analyzer.

**HDL PROGRAM**

**Dataflow Modeling**

```
module full adder (a,b,Cin,Sum,array);
input a,b,Cin;
output Sum, Carry;
assign sum = a^b^Cin;
assign carry = ((a & b)(b & Cin) | (a & Cin));
```

```
end module
```

**BEHAVIORAL MODELING**

```
module Fulladder beh (a, b, cin, sum, Carry)
 input a, b, cin;
 Output sum, Carry;
 reg Sum, Carry;
 always@ (a, or b or cin)
 begin
 sum = a^b ^cin;
 Carry = ((a&b)|(b&cin)|(cin &a));
 end
 end module
```

**GATE LEVEL MODELING**

```
module fulladder str(a,b,cin, Sum, Carry);
input a,b,cin;
 Output Sum,Carry;
 wire t0,t1,t2,t3;
 xor (t0,a,b);
 xor (Sum, t0, cin);
 and (t2,a,b);
 and (t2,b,cin);
 and (t3,a,cin);
 or (Carry, t1,t2,t3);
 end module
```



**TESTBENCH**

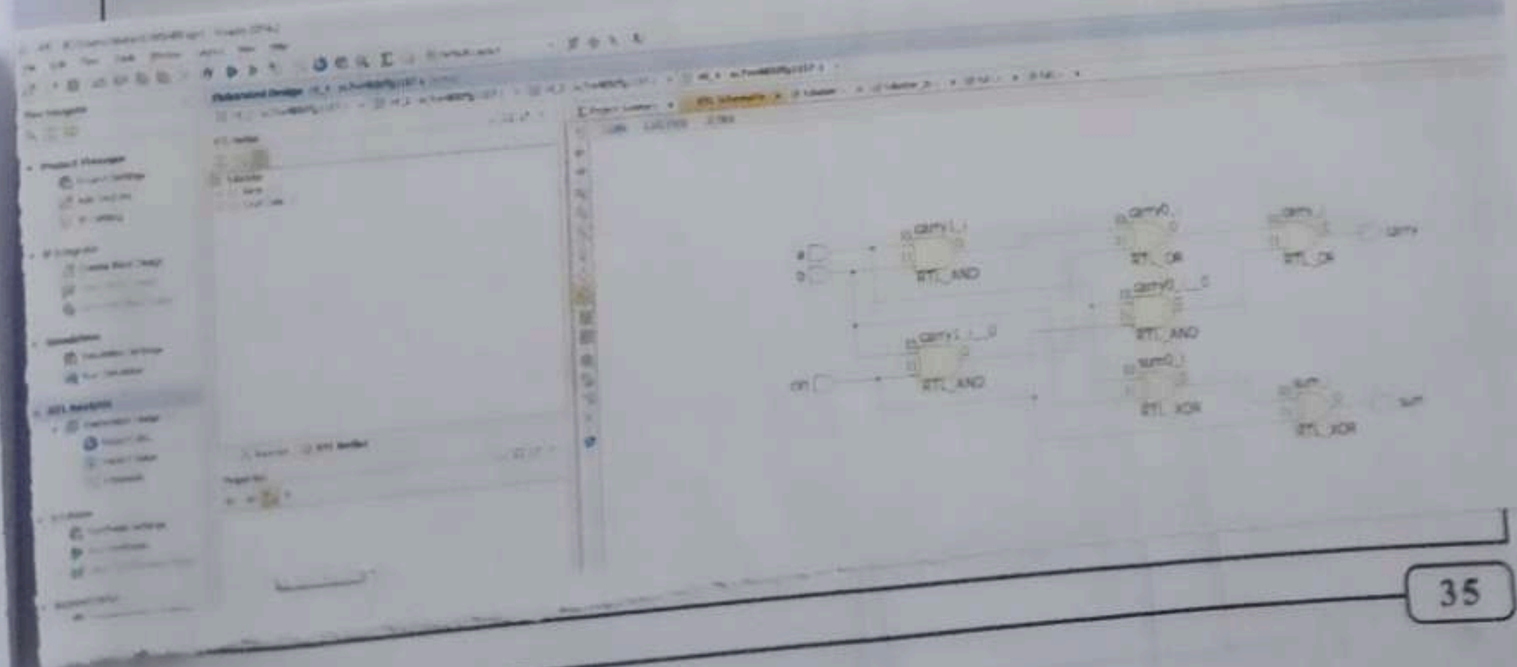
```
module fu
reg a,b,
wire Sum
fulladder

initial b
a=0;b=0;c
a=0 ;b=0
a=0 ;b=1
a=0 ;b=1
a=1 ;b=0
a=1 ;b=0
a=1 ;b=1
a=1 ;b=1
```

**RTL SCHEMATIC**

**TESTBENCH**

```
module fulladder_tb;
reg a,b,Cin
wire Sum,Carry;
fulladder uut (.a(a),.b(b),.Cin(Cin),.Sum(Sum),
                .Carry(Carry));
initial begin
a=0;b=0;c=0;#100;
a=0 ;b=0;c=1;
a=0 ;b=1;c=0;
a=0 ;b=1;c=1;
a=1 ;b=0;c=0;
a=1 ;b=0;c=1;
a=1 ;b=1;c=0;
a=1;b=1;c=1;
end
end module
```

**RTL SCHEMATIC**



## PROCEDURE

1. Click on XILINX VIVADO2014.2
2. Open a new Project and give name to theproject.
3. Open a new source file and give name and specify input and output portnames.
4. Type Verilog code and save thefile.
5. Compile sourcefile.
6. Add test bench to the main module and specify the test vectors init.
7. Simulate the test bench and note down the simulationresults.
8. Synthesize the main module and note down the RTLSchematic.
9. Elaborate the design and assign the pins to thei/os
10. Perform Implementation which internally performs floor planning, placement androuting
11. Generate a bit stream for the givenprogram
12. Dump the bit file on the FPGA and check itsfunctionality
13. Verify the functionality using the Logicanalyzer.

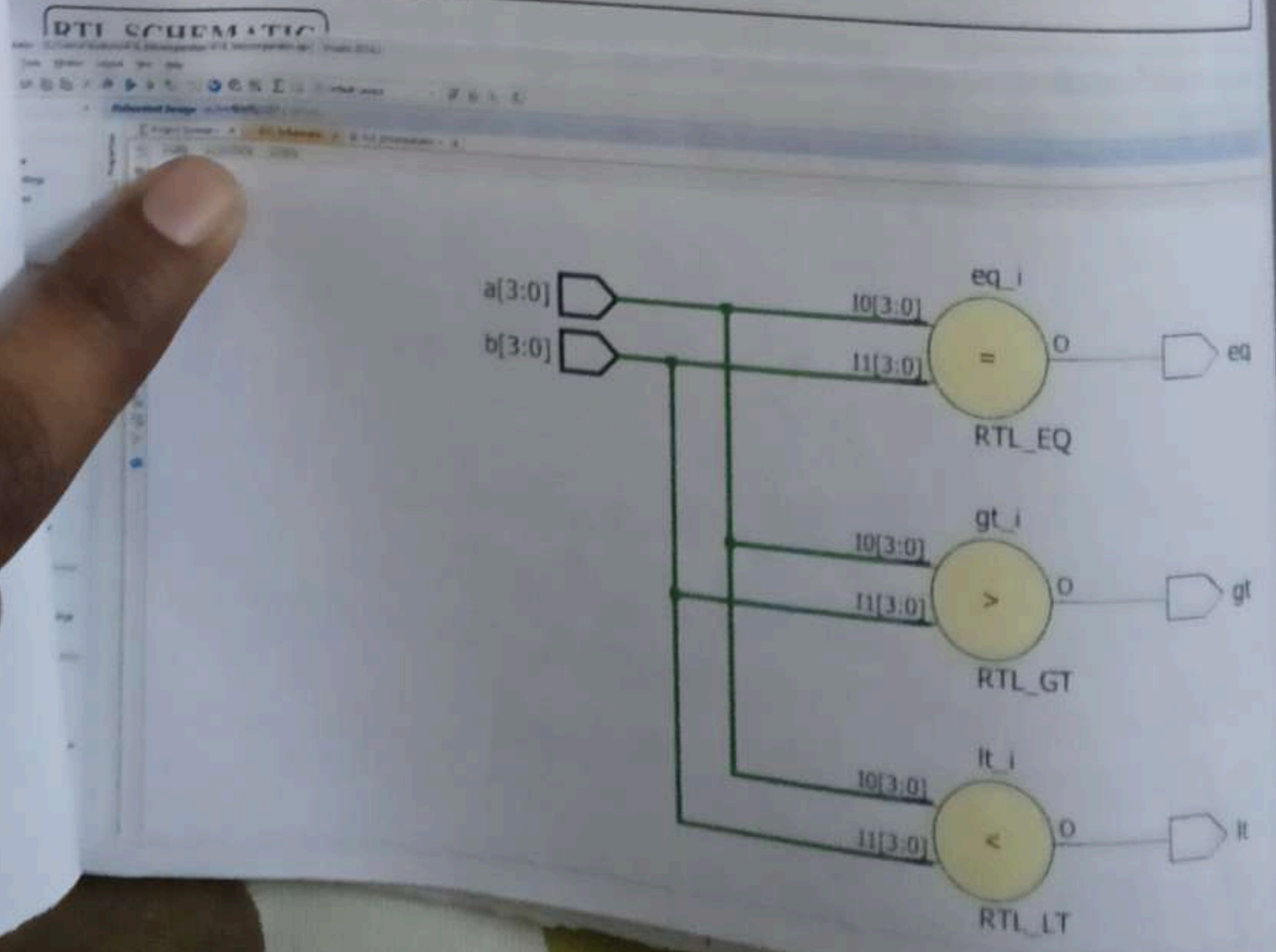
## HDL PROGRAM

```
module Comparator c
  input [3:0]A,
  input [3:0]B,
  Output GT
  output LT,
  Output EQ
  );
  assign GT =(A>B);
  assign LT = (A<B);
  assign EQ = (A==B);
end module
```

TESTBENCH

```
module Comparator_tb;
reg [3:0]A;
reg [3:0]B;
wire GT;
wire LT;
wire EQ;
Comparator uut(.A(A),.B(B),.GT(GT),.LT(LT),
                .EQ(EQ));
initial begin
  A=0; B=0; #100;
  A=0; B=1; #100;
  A=1; B=0; #100;
  A=1; B=1; #100;
end
end module.
```

RTL SCHEMATIC



**HDL PROGRAM**

```
`timescale ins/1ps
module alu_4bit(
    input [3:0]a,b,
    input [2:0] sel
    output reg[3:0] result,
    output reg Carry
); always @(*) begin
    Carry=0;
    case(sel)
      3'b000: begin
          [Carry,result]=a+b;
      end
      3'b001: begin
          [Carry,result]=a-b;
      end
      3'b010: begin
          result= a&b;
      end
      3'b011: begin
          result= a|b;
      end
      3'b100: begin
          result= a^b;
      end
      3'b101: begin
          result= ~a;
      end
      3'b110: begin
      [Carry,result]= a+1;
      end
      3'b111: begin
      [Carry,result]= a-1;
      end
      default: begin
          result = 4'b000;
          Carry=0;
      end
    endcase
end
endmodule
```

**TEST BENCH**

```
`timescale ins/1ps
module alu_4bit_tb;
    reg [3:0] a,b;
    reg [2:0] sel;
    wire [3:0] result;
    wire Carry;
alu_4bit uut( .a(a), .b(b), .sel(sel), .result(result),
          .Carry(Carry));
initial begin
    a=4'b0011; b=4'b0101; sel=3'b000; #10
    a=4'b0110; b=4'b0010; sel=3'b001; #10
    a=4'b1100; b=4'b1010; sel=3'b010; #10
    a=4'b1100; b=4'b1010; sel=3'b011; #10
    a=4'b1100; b=4'b1010; sel=3'b100; #10
    a=4'b1100; b=4'b000; sel=3'b101; #10
    a=4'b1111; b=4'b0000; sel=3'b110; #10
    a=4'b0001; b=4'b0000; sel=3'b111; #10
$finish
end
end module.
```

**HDL PROGRAM**

**SR FLIP FLOP CODE**

```
module SR_FF(input clk,S,R,output reg Q);
    initial Q=0;
    always @ (Posedge clk)
     begin
       if (S==1) && R==0)
          Q <=1;
       else if (S==0 && R==1)
          Q <=0;
       else if (S==0 && R==0)
          Q <= Q;
       else
          Q <=1'bx;
     end
end module
```

**JK FLIP FLOP CODE**

```
module JK_FF (input clk, J,K, output reg Q);
    always @ (Posedge clk) begin
        if (J==0 && K==0)
            Q <= Q;
        else if (J==0 && K==1)
            Q <=0;
        else if (J==1 && K==0)
            Q <=1;
        else
            Q <= ~Q;
    end
end module
```

**D FLIP FLOP CODE**

```
module D_FF (input clk, D, output reg Q);
    initial Q=1;
    always @ (Posedge clk) begin
        Q <= D;
    end
end module
```

**T FLIP FLOP CODE**

```
module T_FF (input clk, T, output reg Q);
    initial Q=0;
    always @ (Posedge clk) begin
        if (T==0)
            Q <= Q;
        else
            Q <= ~Q;
    end
end module
```

**TESTBENCH**

**SR FLIP FLOP CODE**

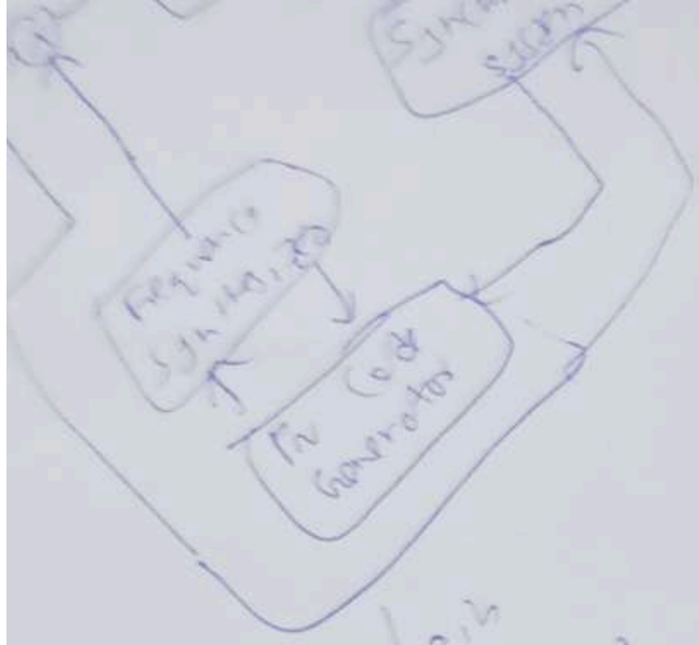
```
module SR-FF_tb();
  reg clk, S, R;
   wire Q;
   SR_FF unt (.clk(clk), .S(S), .R(R), .Q(Q));
   always #5 clk=~clk;
   initial begin
      clk=0; S=0; R=0;
      #7   S=0; R=0;
      #10  S=1; R=0;
      #10  S=0; R=0;
      #10  S=0; R=1;
      #10  S=0; R=0;
      #10  S=1; R=1;
      #10  S=0; R=0;
```

**JK FLIP FLOP CODE**

```
module JK_FF_tb();
   reg clk, J, K;
   wire Q;
   JK_FF unt (.clk(clk), .J(J), .K(K), .Q(Q));
   always #5 clk= ~clk;
   initial begin
      clk=0; J=0; K=0;
      #7   J=1; K=0;
      #10  J=0; K=0;
      #10  J=0; K=1;
      #10  J=1; K=1;
      #10  J=0; K=0;
      #10  J=1; K=1;
      #10  J=0; K=0;
   end
end module.
```

end

end module





**D FLIP FLOPCODE**

```
module D_FF_tb();
  reg clk, D;
  wire Q;
  D_FF uut (.clk(clk), .D(D), .Q(Q));
  always #5 clk = ~clk;
  initial begin
    clk=0; D=0;
    #10
    #7 D=1;
    #10 D=0;
    #10 D=1;
    #10 D=0;
    #10 D=1;
  end
end module
```

**T FLIP FLOPCODE**

```
module T_FF_tb();
  reg clk, T;
  wire Q;
  T_FF uut(.clk(clk), .T(T), .Q(Q));
  always #5 clk = ~clk
  initial begin
    clk=0; T=0;
    #10
    #7 T=1;
    #10 T=0;
    #10 T=1;
    #10 T=0;
    #10 T=1;
    #10 T=1;
    #10 T=0;
  end
end module
```